

图片压缩与解压缩方法说明

BSR1901 的 SDK 支持在图片编码的时候，对图片进行压缩，以减少对 Flash 资源的占用。我们的压缩编码采用的是游程编码（Run-Length Encoding, RLE）压缩算法，这种方式的优点是压缩和解压缩都非常简单。

我们有一个专门的工具 LCD_mcuTool.exe（最新的是 lcd_mcu_tool V0.2.4.9.exe，感谢 WeGui 卢工的技术支持），在 PC 上完成压缩的编码，在 BSR1901 芯片里面完成解码（解压缩）。

数据的表达方法（数据格式）是这样的：

在"RGB565 原始 RLE 压缩"数据储存格式 =

```
{  
    dat0 的数量,dat0h,dat0l,  
    dat1 的数量,dat1h,dat1l,  
    ...,  
    datn 的数量,datnh,datnl,  
    0x00,//0x00 表示结束  
};
```

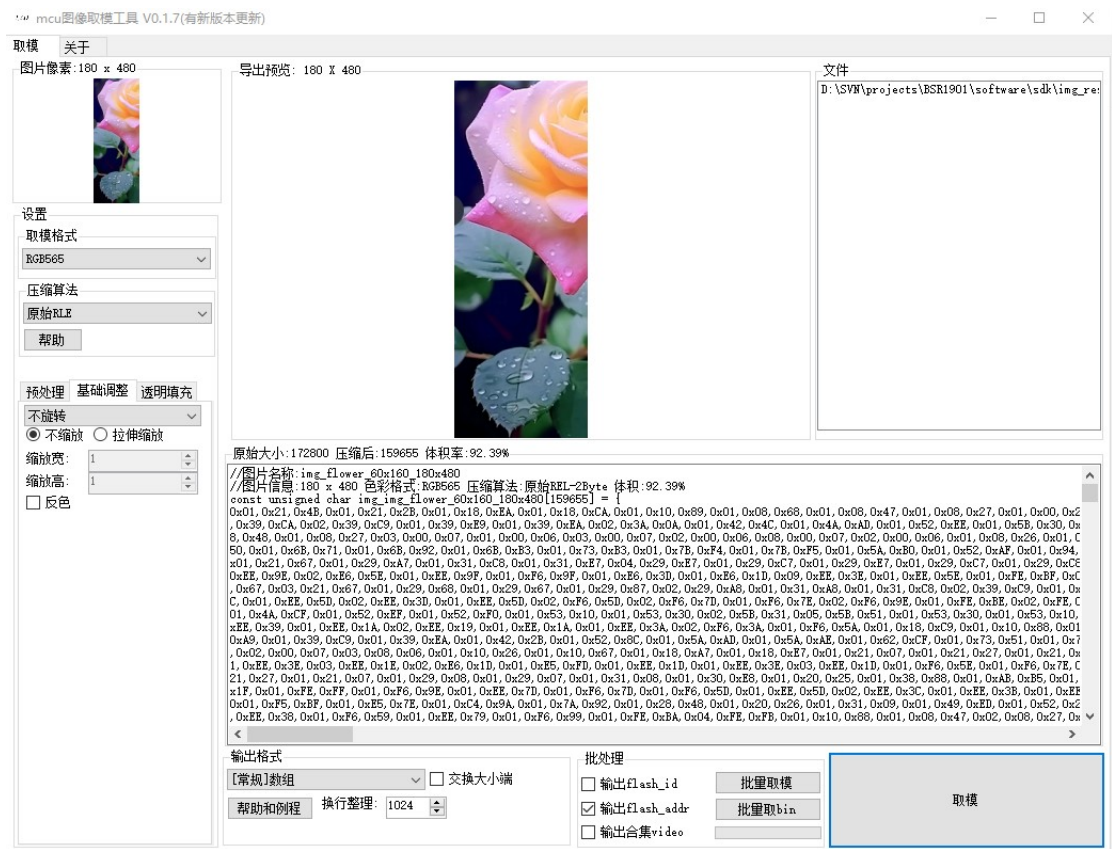
图片 1 示例：

RLE 压缩，体积变成了原始数据的 16.07%，差不多是原来的五分之一。



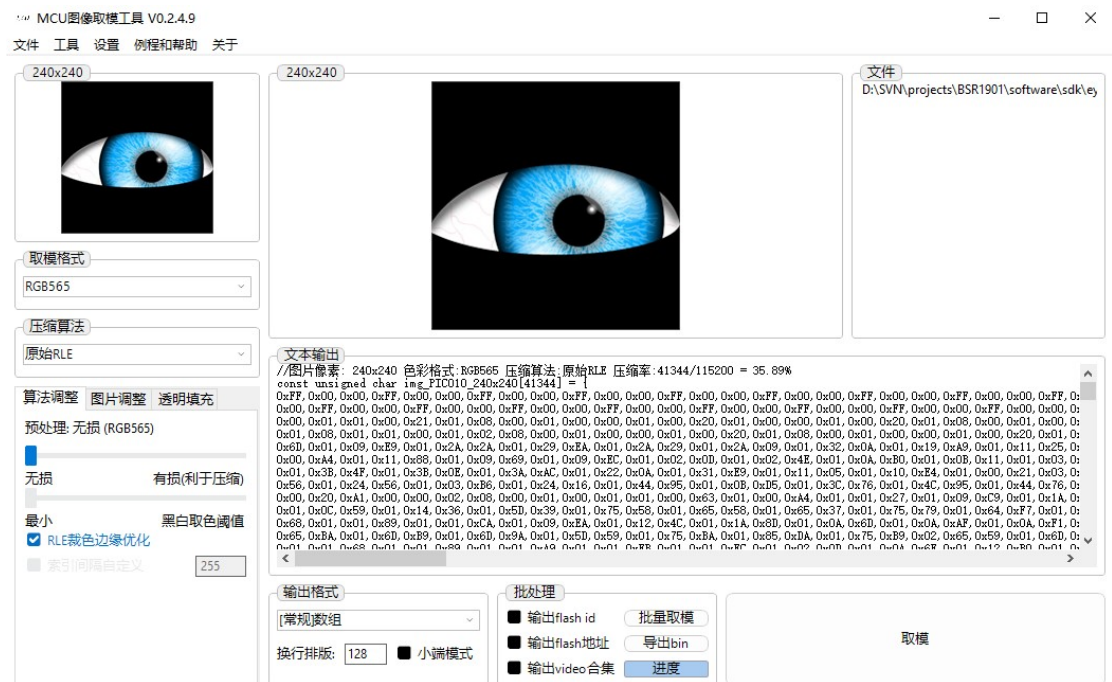
图片 2 示例：

RLE 压缩，体积变成了原始数据的 92.39%，压缩效果变差了。

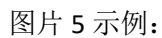


图片 3 示例：

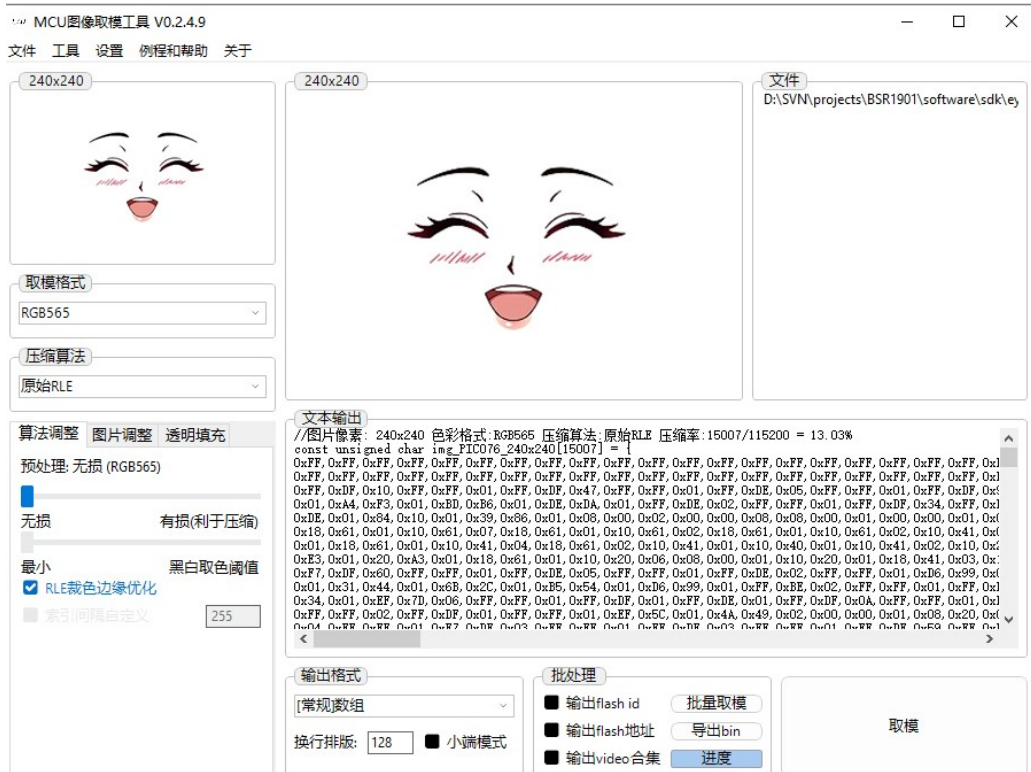
RLE 压缩，体积变成了原始数据的 35.89%，差不多是原来的三分之一。



RLE 压缩，体积变成了原始数据的 59.67%，差不多是原来的二分之一。



RLE 压缩，体积变成了原始数据的 13.03%，差不多是原来的十分之一。



BSR1901 的解压缩方法:

//RGB565 原始 RLE 数据解压例程:

```
void TFT565_draw_arry_rle2_img(uint16_t x, uint16_t y, uint16_t sizex, uint16_t sizey, const uint8_t *arry)
{
    // 设置显示区域（只执行 1 次，不动它）
    Lcd_SetRegion(x, y, x + sizex - 1, y + sizey - 1);
    // 调用 RAM 中的解码函数
    RLE_Decode_Send(arry);
}

// =====
// 配置宏：可自由调整 DMA 触发阈值
// =====
#define RLE_DMA_THRESHOLD 8 // 像素数 > 此值时使用 DMA，否则 CPU 直接发送
__RAM_CODE__ static void RLE_Decode_Send(const uint8_t *arry)
{
    uint8_t count;
    uint8_t dat[2];
    uint16_t color;
    const uint8_t *p = arry;
    for (;;)
    {
        count = *p++;
        if (count == 0) break;
        // 读取 RGB565 颜色
        dat[0] = *p++;
        dat[1] = *p++;
        if (count <= RLE_DMA_THRESHOLD)
        {
            // 短数据：CPU 直接发，比 DMA 快
            do
            {
                SPI_Write16bitData(dat[0]<<8|dat[1]); //LSBF (1<<6) // 1'b0:MSB first (高位在前)
            } while (--count);
        }else
        {
            // 长数据：DMA 批量发送，效率拉满
            color = (uint16_t)dat[0]<<8|dat[1];
            HW_SPI_Tx_DMA_16bit_ColorBlock(&color,count);
        }
    }
}
```

BSR1901 支持 8bit,16bit,24bit,32bit SPI 帧格式，所以一个 16bit(RGB)像素,BSR1901 一条指令就执行完了。

对于相同的，一样的批量数据， BSR1901 的 DMA，可以支持 SRC_NOINC 模式，也是一条指令就行完了。

这种长数据和短数据分开处理的方法，大大提高了解压缩的速度。